

```

package Arvore;

import java.util.ArrayList;

public class ArvoreSimples implements Position{ private int size; private No
root;

public ArvoreSimples(){
    this.size = 0;
    this.root = null;
}

public void replace(No node, Object o) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Erro: Não há nós para acessar");
    }

    if (node == null){
        throw new InvalidPositionExceptionArvore("Erro: Nó nulo");
    }

    node.setElement(o);
}

public Object swapElement(No nodeA, No nodeB) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Erro: Não há nós para acessar");
    }

    if (nodeA == null || nodeB == null){
        throw new InvalidPositionExceptionArvore("Erro: Nó nulo");
    }

    Object salvo = nodeA.getElement();
    nodeA.setElement(nodeB.getElement());
    nodeB.setElement(salvo);

    return salvo;
}

public int depth(No node) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Erro: Não há nós para acessar");
    }

    if (node == null){
        throw new InvalidPositionExceptionArvore("Erro: Nó nulo");
    }
}

```

```

    }

    if (isRoot(node)){
        return 0;
    }

    return 1 + depth(node.getParents());
}

public int height(No node) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Erro: Não há nós para acessar");
    }

    if (node == null){
        throw new InvalidPositionExceptionArvore("Erro: Nó nulo");
    }

    if (isExternal(node)){
        return 0;
    }

    int maxHeight = 0;
    for (int i = 0; i < node.getChildren().size(); i++){
        int conta = height(node.getChildren().get(i));
        maxHeight = Math.max(conta, maxHeight);
    }

    return 1 + maxHeight;
}

public String preorderPrint(No node) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Erro: Não há nós para acessar");
    }

    if (node == null){
        throw new InvalidPositionExceptionArvore("Erro: Nó nulo");
    }

    String resultado = node.getElement().toString();

    for (int i = 0; i < node.getChildren().size(); i++){
        resultado += ", " + preorderPrint(node.getChildren().get(i));
    }
}

```

```

        return resultado;
    }

    public String posorderPrint(Node node) throws InvalidPositionExceptionArvore{
        if (isEmpty()){
            throw new InvalidPositionExceptionArvore("Erro: Não há nós para acessar");
        }

        if (node == null){
            throw new InvalidPositionExceptionArvore("Erro: Nó nulo");
        }

        if (isExternal(node)){
            return node.getElement().toString();
        }

        String resultado = "";

        for (int i = 0; i < node.getChildren().size(); i++){
            resultado += posorderPrint(node.getChildren().get(i)) + ", ";
        }

        resultado += node.getElement().toString();

        return resultado;
    }

    public int size(){
        return size;
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public boolean isRoot(Node node) throws InvalidPositionExceptionArvore{
        if (isEmpty()){
            throw new InvalidPositionExceptionArvore("Erro: Árvore vazia");
        }

        if (root == node && node.getParents() == null){
            return true;
        }

        return false;
    }
}

```

```

public boolean isInternal(No node) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Erro: Árvore vazia");
    }

    if (node.getChildren().size() > 0){
        return true;
    }

    return false;
}

public boolean isExternal(No node) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Erro: Árvore vazia");
    }

    if (node.getChildren().size() == 0){
        return true;
    }

    return false;
}

public void insert(Object o, No node) throws InvalidPositionExceptionArvore{
    if (root == null){
        No root = new No();
        root.setElement(o);
        this.root = root;
        size++;
        return;
    }

    if (node == null){
        throw new InvalidPositionExceptionArvore("Está vazia");
    }

    No novo = new No();
    novo.setElement(o);

    node.setChildren(novo);
    novo.setParents(node);

    size++;
}

```

```

public void remove(No node) throws InvalidPositionExceptionArvore{
    if (isEmpty()){
        throw new InvalidPositionExceptionArvore("Está vazia");
    }

    if (node == null){
        throw new InvalidPositionExceptionArvore("Está vazia");
    }

    if (isInternal(node)){
        No escolhido = node.getChildren().get(0);

        ArrayList<No> filhos = new ArrayList<No>();

        for (int i = 0; i < escolhido.getChildren().size(); i++){
            filhos.add(escolhido.getChildren().get(i));
        }

        node.setElement(escolhido.getElement());
        node.getChildren().remove(0);

        for (int i = 0; i < filhos.size(); i++){
            node.setChildren(filhos.get(i));
            filhos.get(i).setParents(node);
        }

    } else {
        if (isRoot(node)){
            root = null;
        } else {
            node.getParents().getChildren().remove(node);
        }
    }

    size--;
}
}

```